

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

IVH – Projekt
Čítač na maticovém displeji

FPGA / Zynq / VHDL

Obsah

1	Úvod	2
2	Požadované chování systému	2
3	Blokový návrh systému	2
4	Popis architektury	4
4.1	Hlavní FSM řadič	4
4.2	BCD čítače	4
4.3	ROM paměť pro sedmisegmentový kód	4
4.4	Generování 8x8 bitmapy	4
4.5	Buňky cell	4
4.6	Multiplexování displeje	5
5	Použité konstanty	5
6	Simulace	5
6.1	Simulace zobrazování a multiplexu	5
6.2	Simulace FSM řadiče	6
6.3	Simulace ROM paměti	7
6.4	Simulace celého systému	7
7	Implementace na kitu Xilinx Zynq	9
8	Funkční implementace	9
9	Závěr	9

1 Úvod

Cílem projektu bylo navrhnout a implementovat čítač zobrazovaný na maticovém displeji 8x8. Návrh je určen pro FPGA desku se Zynq systémem. Projekt je vytvořen ve VHDL a propojen ve Vivadu pomocí blokového návrhu, který obsahuje hodinový generátor a hlavní komponentu `top`.

Výsledná aplikace postupně zobrazuje číselnou sekvenci, poté přechází do zobrazení obrázku a vlastní animace. Po dokončení celé sekvence se systém vrací zpět na začátek. Zobrazování na displeji je řešeno multiplexováním sloupců.

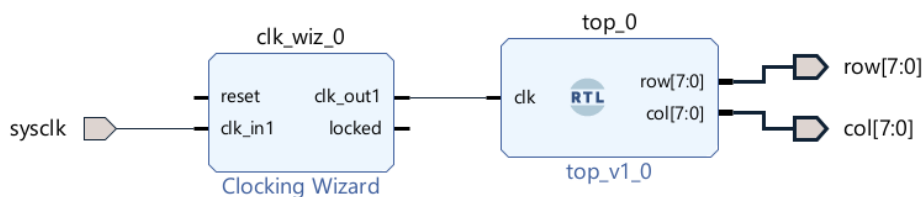
2 Požadované chování systému

Systém pracuje podle následující posloupnosti:

- čítání od 00 do 10 s krokem 1 sekunda,
- čítání od 10 do 99 s krokem 100 ms,
- inverzní odpočet 90, 80, ..., 00 s krokem 500 ms,
- zobrazení obrázku srdce po dobu 5 sekund,
- dvě rotace obrázku doleva,
- vlastní animace tvořená zaplněním displeje, blikáním a spirálovým zhasínáním,
- návrat na začátek sekvence.

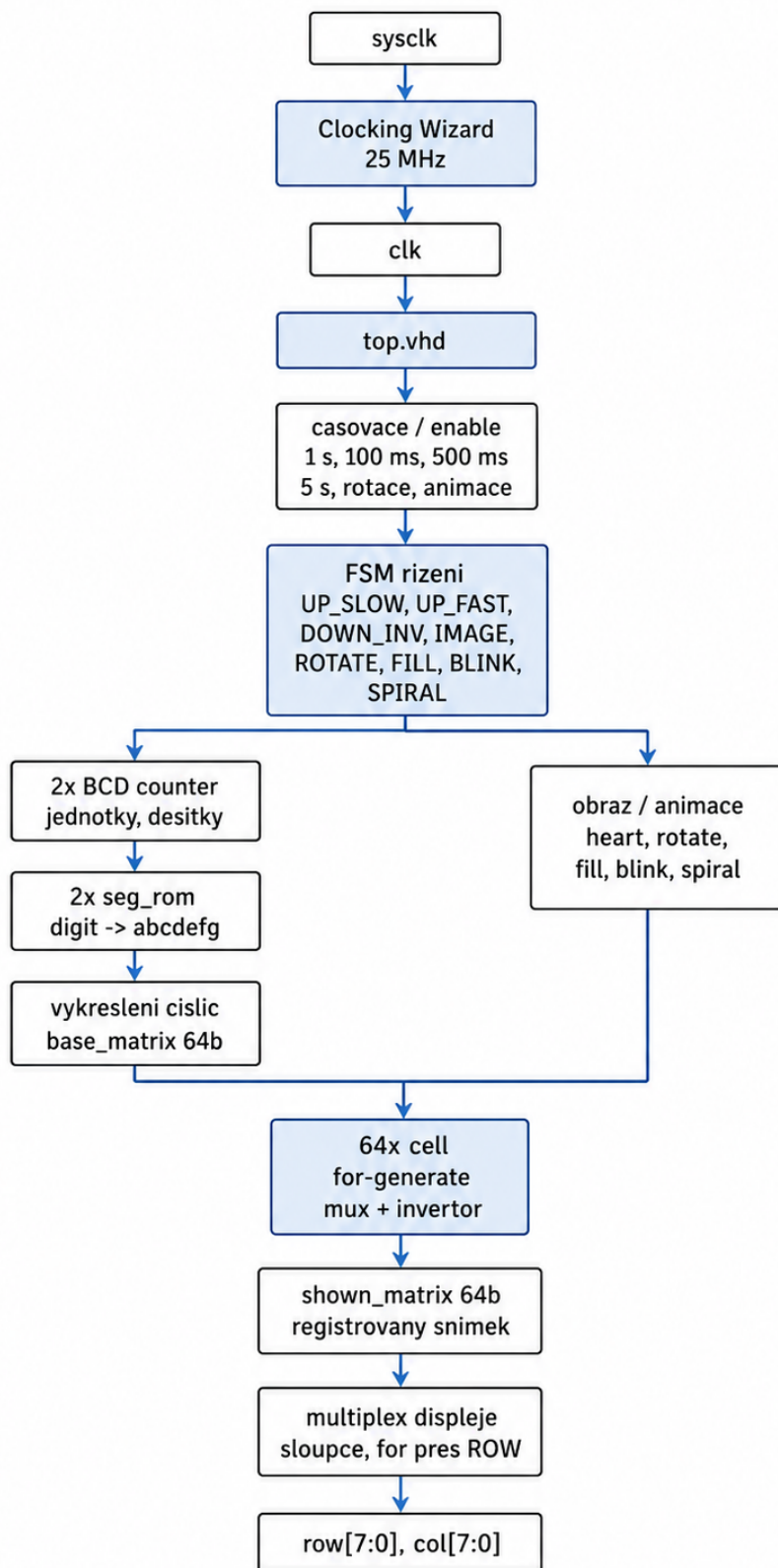
3 Blokový návrh systému

Na nejvyšší úrovni je návrh tvořen blokovým designem ve Vivadu. Vstupní hodinový signál `sysclk` je přiveden do komponenty Clocking Wizard, která generuje hodinový signál 25 MHz pro hlavní VHDL entitu `top`. Z entity `top` jsou vyvedeny výstupy `row[7:0]` a `col[7:0]` pro řízení maticového displeje.



Obrázek 1: Blokový návrh ve Vivadu: Clocking Wizard a hlavní entita `top`.

Vnitřní architektura entity `top` je znázorněna na obrázku 2. Návrh se skládá z časovačů, FSM řadiče, dvojice BCD čítačů, dvou ROM pamětí pro převod číslic na sedmissegmentový kód, generátoru bitmapy, 64 buněk `cell` a zobrazovací části pro multiplexování displeje.



Obrázek 2: Vnitřní architektura systému.

4 Popis architektury

4.1 Hlavní FSM řadič

Chování systému je řízeno konečným automatem. V návrhu jsou použity tyto stavy:

```
S_START_RESET  
S_UP_SLOW  
S_UP_FAST  
S_DOWN_INV  
S_IMAGE  
S_ROTATE  
S_FILL  
S_BLINK  
S_SPIRAL
```

Stav **S_START_RESET** slouží ke krátkému počátečnímu resetu BCD čítačů. Ve stavu **S_UP_SLOW** probíhá pomalé čítání od 00 do 10. Ve stavu **S_UP_FAST** systém pokračuje rychlým čítáním do 99. Po dosažení hodnoty 99 následuje stav **S_DOWN_INV**, kde se zobrazuje inverzní odpočet po desítkách. Poté následuje zobrazení obrázku, jeho rotace a vlastní animace.

4.2 BCD čítače

Pro čítání jsou použity dvě instance komponenty **counter**. První instance reprezentuje jednotky a druhá desítky. Čítače mají vstupy pro reset, směr čítání a povolení čítání. Směr čítání je řízen signálem **dir_cnt**. V normálních stavech systém čítá nahoru, při inverzním odpočtu čítá dolů.

Jednotky jsou v inverzním odpočtu zobrazeny jako nula, aby vznikla posloupnost:

90, 80, 70, ..., 00

4.3 ROM paměť pro sedmisegmentový kód

Pro převod BCD číslice na sedmisegmentový kód je použita komponenta **seg_rom**. Tato ROM paměť převádí hodnoty 0 až 9 na vektor segmentů **a** až **g**. V návrhu jsou použity dvě instance, jedna pro desítky a jedna pro jednotky.

Výstup ROM má tvar:

`seg(6 downto 0) = a b c d e f g`

Hodnota 1 znamená, že daný segment má svítit.

4.4 Generování 8x8 bitmapy

Výstupy ROM jsou převedeny na 64bitovou bitmapu **base_matrix**. Bitmapa reprezentuje displej 8x8 a je uložena po řádcích. Hodnota 1 znamená, že daná LED má svítit.

Číslice jsou vykresleny pomocí pomocných procedur **set_pixel**, **hline**, **vline** a **draw_digit**. Kvůli fyzické orientaci displeje jsou při vykreslování prohozeny horní a dolní segmenty.

4.5 Buňky cell

Výsledný obraz není poslán přímo do zobrazovače, ale nejprve prochází přes 64 instancí entity **cell**. Tyto instance jsou vytvořeny pomocí konstrukce **for-generate**. Každá buňka vybírá, jaký zdroj obrazu bude použit.

Použité režimy buněk jsou:

Režim	Význam
000	BCD obraz
001	BCD obraz pro inverzní odpočet
010	statický obrázek nebo rotace
100	vlastní animace

4.6 Multiplexování displeje

Maticový displej je řízen multiplexováním sloupců. Vždy je aktivní právě jeden sloupec. Sloupce jsou aktivní v logické 1 a řádky jsou aktivní v logické 0. Z tohoto důvodu je výstup `row` negací interní bitmapy.

Jeden sloupec je aktivní přibližně 2 ms. Po průchodu všech osmi sloupců je do zobrazovače vložen nový stabilní snímek `shown_matrix`. Tím se omezuje problikávání při změně zobrazovaného obrazu.

5 Použité konstanty

V návrhu je použit generický parametr:

```
CLK_HZ : integer := 25_000_000
```

Ten určuje frekvenci hodinového signálu. Ve skutečné implementaci je hodnota 25 MHz. V simulacích byla tato hodnota zmenšena, aby bylo možné rychleji ověřit chování systému.

Konstanta	Hodnota	Význam
TICK_1S_MAX	CLK_HZ - 1	časový pulz 1 s
TICK_100MS_MAX	CLK_HZ / 10 - 1	časový pulz 100 ms
TICK_500MS_MAX	CLK_HZ / 2 - 1	časový pulz 500 ms
SCAN_MAX	CLK_HZ / 500 - 1	2 ms na jeden sloupec
IMAGE_5S_MAX	5 * CLK_HZ - 1	zobrazení obrázku 5 s
ROTATE_STEP_MAX	CLK_HZ / 4 - 1	krok rotace 250 ms
FILL_STEP_MAX	CLK_HZ / 16 - 1	rychlost zaplnění
BLINK_STEP_MAX	CLK_HZ / 4 - 1	rychlost blikání
SPIRAL_STEP_MAX	CLK_HZ / 32 - 1	rychlost spirály

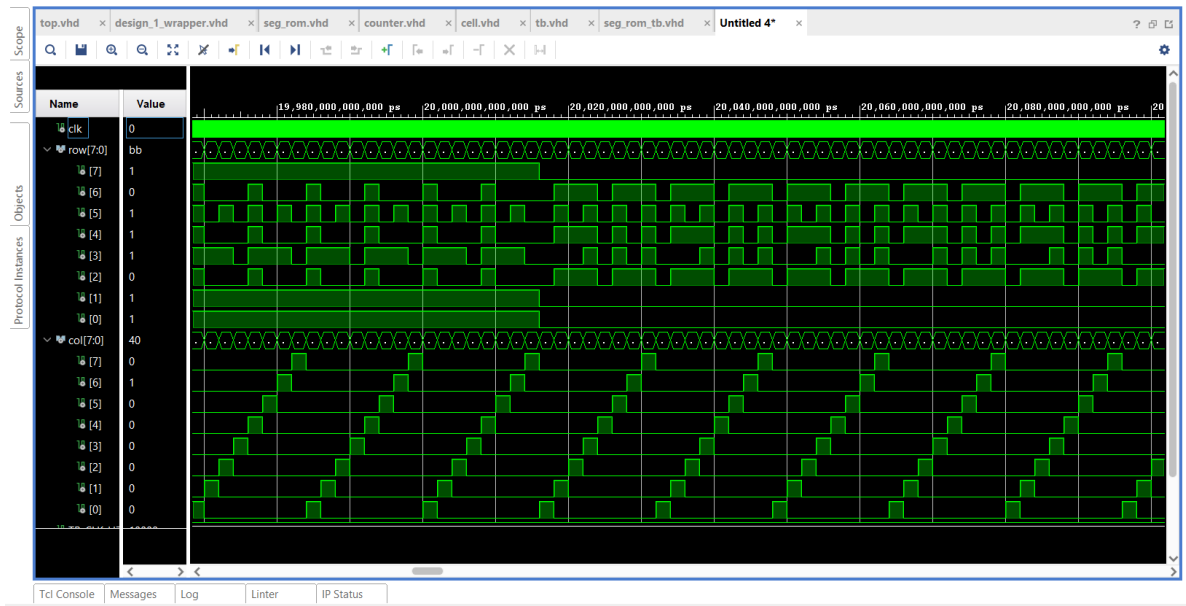
Použití konstant umožňuje snadno měnit časování celého systému a zároveň zrychlit simulace pomocí menší hodnoty CLK_HZ.

6 Simulace

6.1 Simulace zobrazování a multiplexu

Tato simulace ověřuje správnou funkci multiplexování maticového displeje. Ve waveformu jsou sledovány zejména signály `col_idx`, `col`, `row`, `shown_matrix` a pomocný signál `debug_leds`.

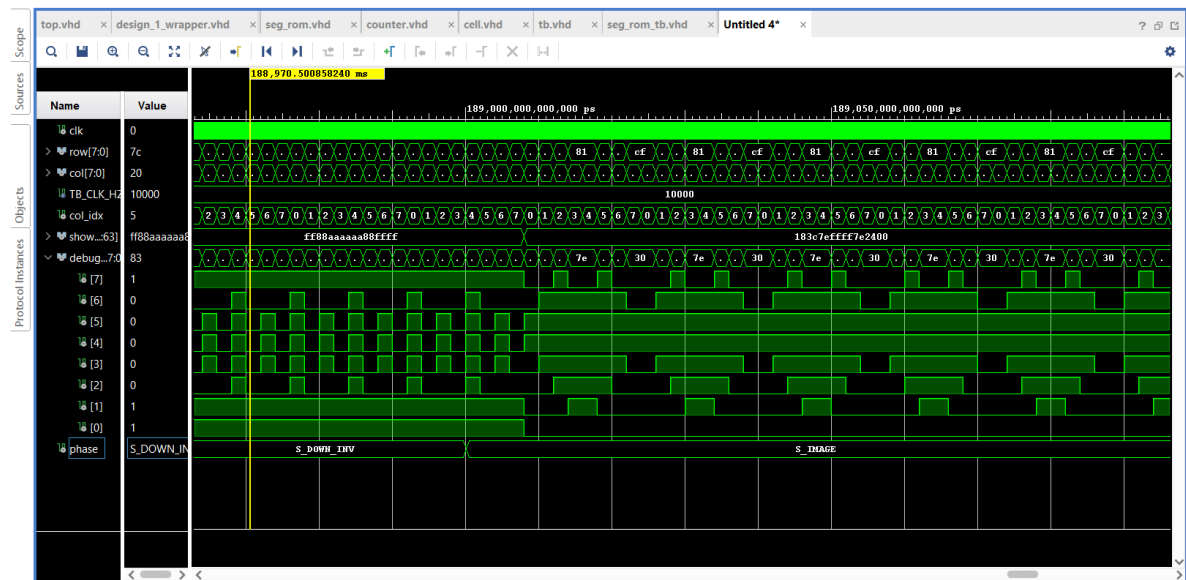
Signál `col_idx` postupně prochází hodnoty 0 až 7. Podle něj je aktivní právě jeden bit výstupu `col`. Signál `debug_leds` ukazuje neinvertovaná data aktuálního sloupce. Výstup `row` je jejich negací, protože řádky displeje jsou aktivní v logické 0.



Obrázek 3: Simulace zobrazování a multiplexování displeje.

6.2 Simulace FSM řadiče

Simulace FSM řadiče ověřuje průchod všemi hlavními stavy aplikace. Ve waveformu jsou sledovány zejména signály `phase`, `digit_tens`, `digit_ones`, `tick_1s`, `tick_100ms`, `tick_500ms`, `cell_mode` a `cell_invert`.

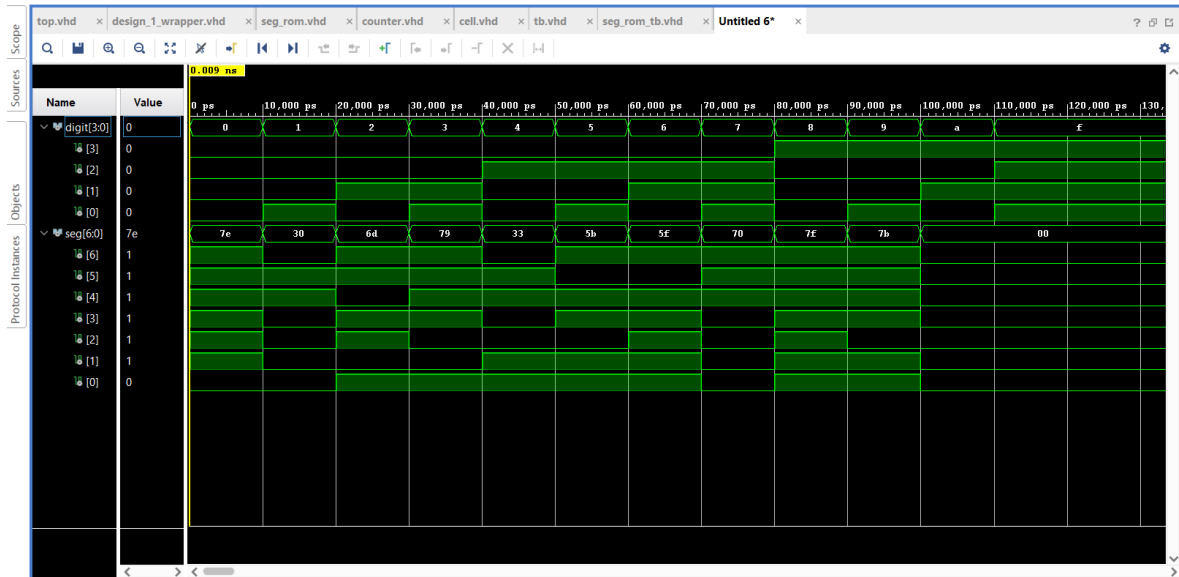


Obrázek 4: Simulace FSM řadiče.

Ve stavu `S_UP_SLOW` jsou čítače povolovány pulzem `tick_1s`. Po dosažení hodnoty 10 přechází systém do stavu `S_UP_FAST`, kde se používá rychlejší pulz `tick_100ms`. Po dosažení hodnoty 99 následuje inverzní odpočet ve stavu `S_DOWN_INV`. V tomto stavu je aktivní signál `cell_invert`. Následně systém přechází do stavů pro obrázek, rotaci a animaci. FSM řadič tedy odpovídá požadovanému chování.

6.3 Simulace ROM paměti

Simulace ROM paměti byla provedena pomocí testbenche `seg_rom_tb`. Testbench postupně nastavuje vstup `digit` na hodnoty 0 až 9 a pomocí příkazů `assert` kontroluje, zda výstup `seg` odpovídá očekávanému sedmisegmentovému kódu. Dále jsou ověřeny neplatné hodnoty 10 a 15, pro které ROM vrací nulový výstup.



Obrázek 5: Simulace ROM paměti `seg_rom`.

Simulace proběhla bez chybových hlášení `assert`. To potvrzuje, že ROM paměť správně převádí BCD číslice na sedmisegmentový kód.

6.4 Simulace celého systému

Simulace celého systému byla provedena nad entitou `top`. Pro účely simulace byla snížena hodnota generického parametru `CLK_HZ`, aby bylo možné projít větší část chování systému v rozumném čase.

Ve waveformu byly sledovány signály `phase`, `digit_tens`, `digit_ones`, `disp_tens`, `disp_ones`, `seg_tens`, `seg_ones`, `cell_mode`, `cell_invert`, `shown_matrix`, `row` a `col`.

dosažení poslední pozice animace se aktivuje reset čítačů a FSM přechází zpět do stavu S_UP_SLOW. Tím začíná nová iterace programu, ve které se znovu spouští čítání od hodnoty 00. Simulace potvrzuje, že systém po dokončení animace nezůstane stát, ale správně se vrací na začátek požadované smyčky.

7 Implementace na kitu Xilinx Zynq

Po syntéze, implementaci a nahrání bitstreamu na kit Xilinx Zynq byla ověřena funkce maticového displeje. Displej zobrazuje nejprve číselnou sekvenci, poté inverzní odpočet, obrázek srdce a vlastní animaci. Zobrazování je stabilní a multiplexování sloupců probíhá správně.

8 Funkční implementace

Video s ukázkou funkční implementace je dostupné na následujícím odkazu:

[Odkaz na video s funkční implementací](#)

Před odevzdáním byl odkaz ověřen v anonymním okně prohlížeče, aby bylo zajištěno, že je video dostupné i bez přihlášení.

9 Závěr

V projektu byl vytvořen systém pro řízení maticového displeje 8x8. Návrh obsahuje FSM řadič, dvojici BCD čítačů, ROM paměť pro převod číslic, generování bitmapy, 64 instancí buněk `cell` a zobrazovací multiplex. Systém splňuje požadovanou posloupnost čítání, inverzního odpočtu, zobrazování obrázku a animace.